

Orbiter autour de... rien?

Stabilité des points de Lagrange et cinématique dans leur voisinage

ABDELOUAHAB Yannis, DUPLOUY Paul-Emmanuel

SCEI 21691

Session 2026

Sommaire

1. Objectif

2. Dynamique au voisinage des points L1 et L2

2.1 Définition du système, points d'équilibres

2.2 Résolution du système linéarisé

2.3 Schéma numérique pour le système non-linéaire

3. Globalisation des variétés des orbites autour de L1 et L2

3.1 Existence d'orbites périodiques

3.2 Introduction à la théorie de Floquet

3.3 Vecteurs propres de la matrice de Monodromie

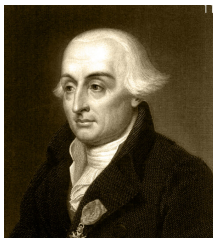
3.4 Calcul numérique des variétés

4. Conclusion

Introduction



(a) Leonhard Euler



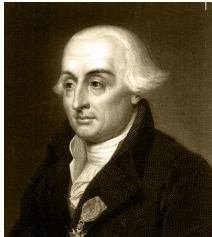
(b) Joseph-Louis
Lagrange

Figure 1.1

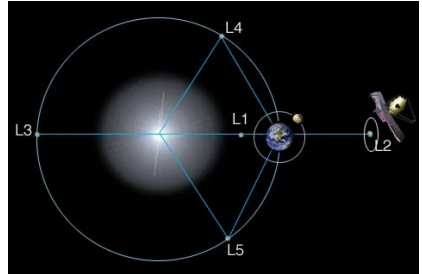
Introduction



(a) Leonhard Euler



(b) Joseph-Louis Lagrange



(c) Orbite du James Webb Space Telescope

Figure 1.1

Objectif

Objectif :

Comprendre en quoi les points de Lagrange L_1 et L_2 sont intéressants pour les missions spatiales d'observation, et leur utilité dans la création de trajectoires moins demandeuses en carburant.

Sommaire

1. Objectif

2. Dynamique au voisinage des points L1 et L2

2.1 Définition du système, points d'équilibres

2.2 Résolution du système linéarisé

2.3 Schéma numérique pour le système non-linéaire

3. Globalisation des variétés des orbites autour de L1 et L2

3.1 Existence d'orbites périodiques

3.2 Introduction à la théorie de Floquet

3.3 Vecteurs propres de la matrice de Monodromie

3.4 Calcul numérique des variétés

4. Conclusion

Choix du référentiel

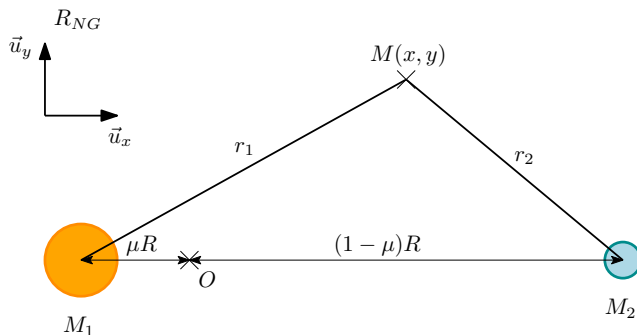


Figure 2.1.1 – Plan $(O, \vec{u}_x, \vec{u}_y)$ dans le référentiel barycentrique R_{NG} en co-rotation avec le système Soleil–Terre.

Avec $\mu := \frac{M_2}{M_1 + M_2}$ le *coefficient de masse* du système.

Coordonnées des points de Lagrange

Point	Coordonnée en x	Coordonnée en y
L1	$((1 - \mu) - (\frac{\mu}{3})^{\frac{1}{3}})R$	0
L2	$((1 - \mu) + (\frac{\mu}{3})^{\frac{1}{3}})R$	0
L3	$(-1 - \frac{5\mu}{12})R$	0
L4	$((\frac{1}{2} - \mu)R)$	$\frac{\sqrt{3}}{2}R$
L5	$((\frac{1}{2} - \mu)R)$	$-\frac{\sqrt{3}}{2}R$

Figure 2.1.2 – Coordonnées des points d'équilibres dans le référentiel tournant, dimensionnalisées.

Représentation des points de Lagrange

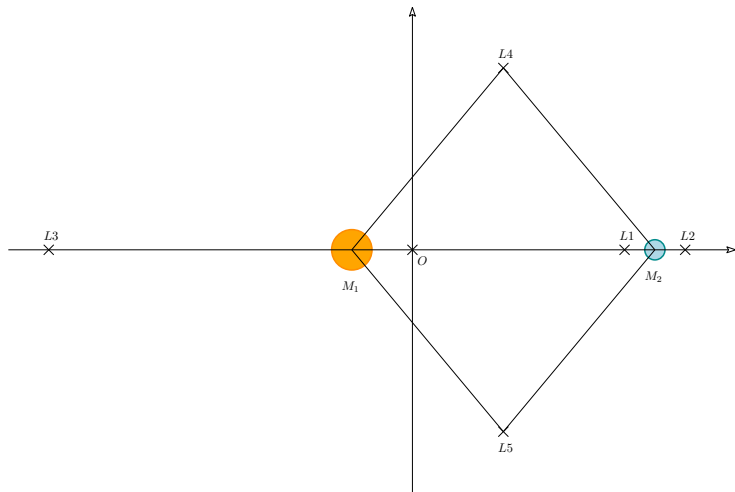


Figure 2.1.3 – Positions des points de Lagrange dans le plan d'étude.

Linéarisation des équations du mouvement

Les équations du mouvements sont

$$\begin{cases} \dot{x} = v_x \\ \dot{y} = v_y \\ v_x = \dot{x} = 2\omega y - \frac{1}{m}\bar{U}_x \\ v_y = \dot{y} = -2\omega x - \frac{1}{m}\bar{U}_y \end{cases}$$

Ce qui se linéarise en

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{1}{m}\bar{U}_{xx}^i & -\frac{1}{m}\bar{U}_{xy}^i & 0 & 2\omega \\ -\frac{1}{m}\bar{U}_{yx}^i & -\frac{1}{m}\bar{U}_{yy}^i & -2\omega & 0 \end{bmatrix} =: A \quad (1)$$

où $\bar{U}_{wz}^i = \frac{\partial^2 \bar{U}}{\partial w \partial z}(x_{L_i}, y_{L_i})$

Résolution

Le calcul du polynôme caractéristique χ_A de A nous donne :

$$A = P \begin{bmatrix} \lambda & 0 & 0 & 0 \\ 0 & -\lambda & 0 & 0 \\ 0 & 0 & i\nu & 0 \\ 0 & 0 & 0 & -i\nu \end{bmatrix} P^{-1}, \text{ avec } \begin{cases} \lambda = \omega \sqrt{1 + 2\sqrt{7}} \\ \nu = \omega \sqrt{2\sqrt{7} - 1} \end{cases}$$

Résolution

Le calcul du polynôme caractéristique χ_A de A nous donne :

$$A = P \begin{bmatrix} \lambda & 0 & 0 & 0 \\ 0 & -\lambda & 0 & 0 \\ 0 & 0 & i\nu & 0 \\ 0 & 0 & 0 & -i\nu \end{bmatrix} P^{-1}, \text{ avec } \begin{cases} \lambda = \omega\sqrt{1+2\sqrt{7}} \\ \nu = \omega\sqrt{2\sqrt{7}-1} \end{cases}$$

On en déduit l'expression explicite des solutions dans le plan :

$$\mathcal{S}_{lin}^{pos} = \{t \rightarrow \begin{bmatrix} C_1 e^{\lambda t} + C_2 e^{-\lambda t} + (C_3 + C_4) \cos(\nu t) \\ -\sigma C_1 e^{\lambda t} + \sigma C_2 e^{-\lambda t} + \tau (C_3 + C_4) \sin(\nu t) \end{bmatrix}, (C_1, C_2, C_3, C_4) \in \mathbb{R}^3\} \quad (2)$$

où $\sigma > 0$ et $\tau < 0$ sont des constantes.

Simulation de la dégénérescence autour de L1

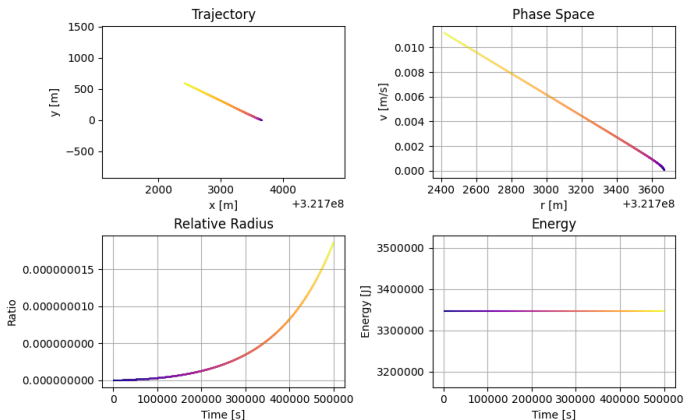


Figure 2.3.1 – Simulation numérique par schéma de Runge-Kutta 4 de la dégénérescence de l'orbite autour de L1.

Sommaire

1. Objectif

2. Dynamique au voisinage des points L1 et L2

2.1 Définition du système, points d'équilibres

2.2 Résolution du système linéarisé

2.3 Schéma numérique pour le système non-linéaire

3. Globalisation des variétés des orbites autour de L1 et L2

3.1 Existence d'orbites périodiques

3.2 Introduction à la théorie de Floquet

3.3 Vecteurs propres de la matrice de Monodromie

3.4 Calcul numérique des variétés

4. Conclusion

Obtention d'une orbite

Forme générale des solutions dans l'espace de phase

$$\mathcal{S}_{lin} = \left\{ t \rightarrow \begin{bmatrix} C_1 e^{\lambda t} + C_2 e^{-\lambda t} + R \cos(\nu t - \varphi) \\ -\sigma(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \tau R \sin(\nu t - \varphi) \\ \lambda(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \nu R \sin(\nu t - \varphi) \\ -\lambda\sigma(C_1 e^{\lambda t} + C_2 e^{-\lambda t}) + \nu\tau R \cos(\nu t - \varphi) \end{bmatrix}, (C_1, C_2, R, \varphi) \in \mathbb{R}^4 \right\} \quad (3)$$

Obtention d'une orbite

Forme générale des solutions dans l'espace de phase

$$\mathcal{S}_{lin} = \{t \rightarrow \begin{bmatrix} C_1 e^{\lambda t} + C_2 e^{-\lambda t} + R \cos(\nu t - \varphi) \\ -\sigma(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \tau R \sin(\nu t - \varphi) \\ \lambda(C_1 e^{\lambda t} - C_2 e^{-\lambda t}) + \nu R \sin(\nu t - \varphi) \\ -\lambda\sigma(C_1 e^{\lambda t} + C_2 e^{-\lambda t}) + \nu\tau R \cos(\nu t - \varphi) \end{bmatrix}, (C_1, C_2, R, \varphi) \in \mathbb{R}^4\} \quad (3)$$

On en déduit qu'une orbite de rayon R à pour condition initiale $\begin{cases} x_0 = R \\ y_0 = 0 \\ v_{x0} = 0 \\ v_{y0} = \nu\tau R \end{cases}$

Optimisation numérique des caractéristiques de l'orbite

Conditions initiales proposées :

- $x_0 = 1000 \text{ km}$
- $y_0 = 0 \text{ km}$
- $v_{x0} = 0 \text{ m.s}^{-1}$
- $v_{y0} = 23,7 \text{ m.s}^{-1}$

car $\nu\tau = -2,37 \cdot 10^{-5} \text{ s}^{-1}$

Optimisation numérique des caractéristiques de l'orbite

Conditions initiales proposées :

- $x_0 = 1000 \text{ km}$
- $y_0 = 0 \text{ km}$
- $v_{x0} = 0 \text{ m.s}^{-1}$
- $v_{y0} = 23,7 \text{ m.s}^{-1}$

car $\nu\tau = -2,37 \cdot 10^{-5} \text{ s}^{-1}$

Orbite corrigée par correction itérative :

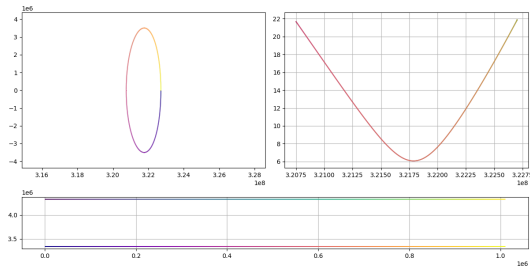


Figure 3.1.1 – Simulation d'une orbite, notée γ , avec les conditions initiales $x_0 = 1000 \text{ km}$, $y_0 = 0 \text{ km}$,

$v_{x0} = 0 \text{ m.s}^{-1}$, et

$v_{y0} = -21.898965265005835 \text{ m.s}^{-1}$.

Illustration de la correction itérative

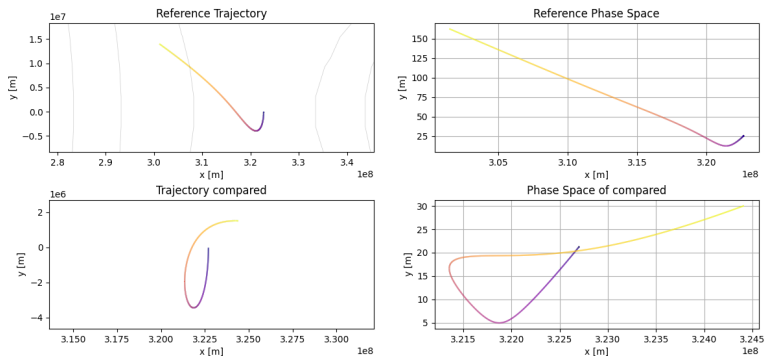


Figure 3.1.2 – Illustration de la méthode de correction itérative par dichotomie (Code python en annexe ??).

Une petite définition...

Définition : Matrice fondamentale

Soit $(y_1, \dots, y_n)$ un système fondamentale de solutions d'une équation différentielle. Une matrice fondamentale de cette équation différentielle est l'application $t \mapsto \Phi(t)$ où $\Phi(t)$ est la matrice dont les colonnes sont les vecteurs $(y_1(t), \dots, y_n(t))$.

Si Φ et Ψ sont deux matrices fondamentales de la même équation différentielle, alors

$$\exists M \in GL_n(\mathbb{C}), \Phi = \Psi M$$

Remarque : Un résultat utile

Si y est une solution de l'équation différentielle, alors $\forall t \in \mathbb{R}, y(t) = \Phi(t)y_0$ où Φ est la matrice fondamentale principale.

...et deux autres...

Définition : Le flot

On définit le flot comme l'application $\eta : \begin{array}{ccc} \mathbb{R} \times \mathbb{R}^4 & \rightarrow & \mathbb{R}^4 \\ (t, x_0) & \mapsto & x(t) \end{array}$ où x correspond à l'unique solution au problème de Cauchy de condition initiale x_0 .
On peut remarquer qu'on a alors la relation

$$\forall x_0 \in \mathbb{R}^4, \forall t \in \mathbb{R}, \eta(t, x_0) = \Phi(t)x_0$$

...et deux autres...

Définition : Le flot

On définit le flot comme l'application $\eta : \begin{array}{ccc} \mathbb{R} \times \mathbb{R}^4 & \rightarrow & \mathbb{R}^4 \\ (t, x_0) & \mapsto & x(t) \end{array}$ où x correspond à l'unique solution au problème de Cauchy de condition initiale x_0 .
On peut remarquer qu'on a alors la relation

$$\forall x_0 \in \mathbb{R}^4, \forall t \in \mathbb{R}, \eta(t, x_0) = \Phi(t)x_0$$

Définition : Matrice de Monodromie

Soit une équation différentielle de la forme $X' = A(t)X$ où A est à coefficients T -périodiques. On appelle *matrice de Monodromie* la matrice $M = \Phi(T)$ où Φ est la matrice fondamentale principale du système.

Sensibilité du système aux conditions initiales

Le flot vérifie pour $t \in \mathbb{R}$ fixé, $x_0 \in \mathbb{R}^4$ et δx_0 une petite variation,

$$\eta(t, x_0 + \delta x_0) = \Phi(t)(x_0 + \delta x_0) = \eta(t, x_0) + \Phi(t)\delta x_0 + o_{\delta x_0 \rightarrow 0}(\delta x_0)$$

Sensibilité du système aux conditions initiales

Le flot vérifie pour $t \in \mathbb{R}$ fixé, $x_0 \in \mathbb{R}^4$ et δx_0 une petite variation,

$$\eta(t, x_0 + \delta x_0) = \Phi(t)(x_0 + \delta x_0) = \eta(t, x_0) + \Phi(t)\delta x_0 + \underset{\delta x_0 \rightarrow 0}{o}(\delta x_0)$$

dont on déduit le

Lien matrice de Monodromie/sensibilité aux C.I.

$$\forall k \in \mathbb{N}^*, \delta x(kT) = M^k \delta x_0 \quad (4)$$

En quête de la Monodromie

Méthode numérique pour calculer M

1. Calcul du linéarisé en chaque point de l'orbite $A : t \mapsto Jf(\gamma(t))$.
2. Intégration de $\Phi(0) = I_n$ par le linéarisé le long de l'orbite, pendant une période T .

En quête de la Monodromie

Méthode numérique pour calculer M

1. Calcul du linéarisé en chaque point de l'orbite $A : t \mapsto Jf(\gamma(t))$.
2. Intégration de $\Phi(0) = I_n$ par le linéarisé le long de l'orbite, pendant une période T .

Linéarisé :

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{\bar{U}_{xx}(\gamma(t))}{m} & -\frac{\bar{U}_{xy}(\gamma(t))}{m} & 0 & 2\omega \\ -\frac{\bar{U}_{xy}(\gamma(t))}{m} & -\frac{\bar{U}_{yy}(\gamma(t))}{m} & -2\omega & 0 \end{bmatrix}$$

En quête de la Monodromie

Méthode numérique pour calculer M

1. Calcul du linéarisé en chaque point de l'orbite $A : t \mapsto Jf(\gamma(t))$.
2. Intégration de $\Phi(0) = I_n$ par le linéarisé le long de l'orbite, pendant une période T .

Linéarisé :

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{\bar{U}_{xx}(\gamma(t))}{m} & -\frac{\bar{U}_{xy}(\gamma(t))}{m} & 0 & 2\omega \\ -\frac{\bar{U}_{xy}(\gamma(t))}{m} & -\frac{\bar{U}_{yy}(\gamma(t))}{m} & -2\omega & 0 \end{bmatrix}$$

Orbite de référence :

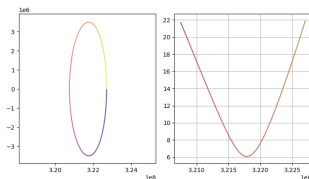


Figure 3.3.1 – Trajectoire de l'orbite de référence γ .

Calcul numérique de M

$$M = \begin{bmatrix} 1.62 \cdot 10^3 & -2.48 \cdot 10^2 & 1.54 \cdot 10^8 & 7.52 \cdot 10^7 \\ -7.92 \cdot 10^2 & 1.22 \cdot 10^2 & -7.52 \cdot 10^7 & -3.69 \cdot 10^7 \\ 1.28 \cdot 10^{-2} & -1.96 \cdot 10^{-3} & 1.22 \cdot 10^3 & 5.96 \cdot 10^2 \\ -6.01 \cdot 10^{-3} & 9.21 \cdot 10^{-4} & -5.72 \cdot 10^2 & -2.79 \cdot 10^2 \end{bmatrix}$$

$$\text{Sp}(M) = \underbrace{\{2.68 \cdot 10^3\}}_{=:\lambda_i}, \underbrace{\{3.74 \cdot 10^{-4}\}}_{=:\lambda_s}, 9.97 \cdot 10^{-1}, 1.00\}$$

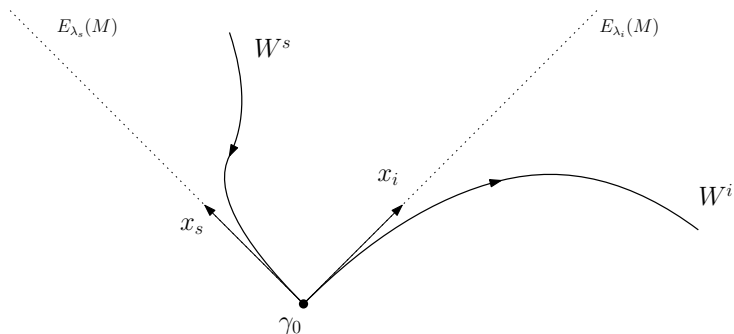
$$P = \begin{bmatrix} -8.98 \cdot 10^{-1} & -8.98 \cdot 10^{-1} & -2.82 \cdot 10^{-1} & -2.84 \cdot 10^{-1} \\ 4.40 \cdot 10^{-1} & -4.40 \cdot 10^{-1} & 9.59 \cdot 10^{-1} & -9.59 \cdot 10^{-1} \\ -7.11 \cdot 10^{-6} & 7.11 \cdot 10^{-6} & 1.55 \cdot 10^{-6} & -1.55 \cdot 10^{-6} \\ 3.34 \cdot 10^{-6} & 3.34 \cdot 10^{-6} & 6.06 \cdot 10^{-6} & 6.10 \cdot 10^{-6} \end{bmatrix}$$

Figure 3.3.2 – Matrice de Monodromie calculée numériquement (cf. Annexe ??) pour l'orbite de référence γ .

Les directions propres

Sensibilités aux variations de C.I. dans une direction propre pour le linéarisé :

$$\forall k \in \mathbb{N}^*, \delta x(kT) = \lambda_u^k x_u, \text{ où } u \in \{s, i\}$$



Calcul numérique des variétés

On utilise comme condition initiale $\gamma_0 \pm \varepsilon \mathcal{N} x_i$ où $\varepsilon = 10^{-6}$ et $\mathcal{N}$ est la dimensionalisation du problème pour l'espace des phases (*).

Variété instable

On intègre le système non-linéaire 9 en prenant comme condition initiale celle précédente.

Calcul numérique des variétés

On utilise comme condition initiale $\gamma_0 \pm \varepsilon \mathcal{N} x_i$ où $\varepsilon = 10^{-6}$ et $\mathcal{N}$ est la dimensionalisation du problème pour l'espace des phases (*).

Variété instable

On intègre le système non-linéaire 9 en prenant comme condition initiale celle précédente.

Variété stable

On intègre le système non-linéaire en temps décroissant. Cela revient à intégrer

$$X' = -f(X)$$

avec la condition initiale ci-dessus.

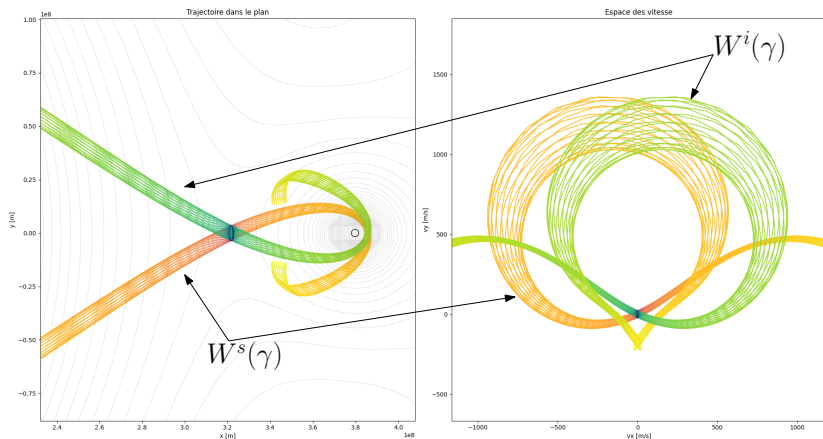


Figure 3.4.1 – Projections dans le plan (x, y) et dans le plan (v_x, v_y) des variétés de l'orbite de référence γ , intégrées numériquement en Python.

Sommaire

1. Objectif

2. Dynamique au voisinage des points L1 et L2

2.1 Définition du système, points d'équilibres

2.2 Résolution du système linéarisé

2.3 Schéma numérique pour le système non-linéaire

3. Globalisation des variétés des orbites autour de L1 et L2

3.1 Existence d'orbites périodiques

3.2 Introduction à la théorie de Floquet

3.3 Vecteurs propres de la matrice de Monodromie

3.4 Calcul numérique des variétés

4. Conclusion

Des applications concrètes

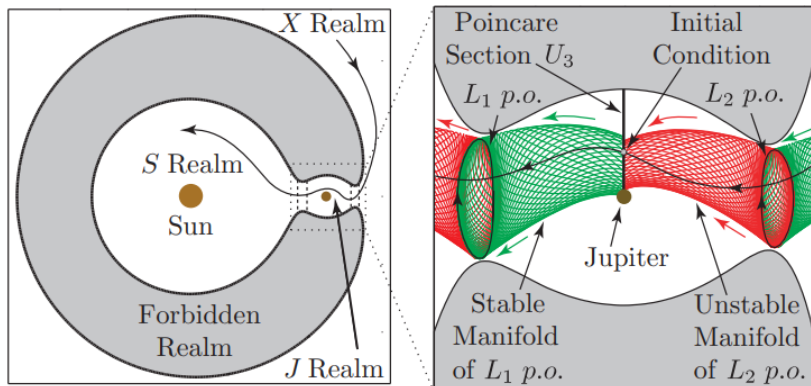


Figure 4.1 – Illustration d'une méthode pour créer des orbites de transfert.

Des applications concrètes

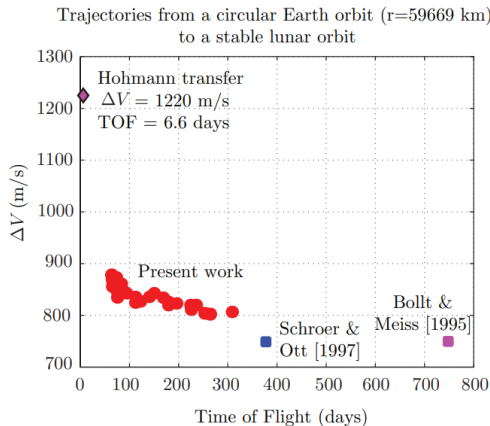


Figure 4.2 – Comparaison des orbites de transfert utilisant les variétés aux orbites de transfert de type *Hohmann*.

FIN

ANNEXES

Calcul des points d'équilibre dans le référentiel tournant

Calcul pour L1

On a

$$\omega^2 x - \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x - \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} - \frac{1 - \mu}{(x + \mu R)^2} + \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \varepsilon + (1 - \mu)R$ on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{(\varepsilon + R)^2} + \frac{\mu}{\varepsilon^2} = 0$$

On peut alors procéder à un développement limité en $\frac{\varepsilon}{R}$. Ce dernier est ici supposé, motivé par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\varepsilon \rightarrow 0$. Le résultat final permettra, ou non, de légitimiser notre supposition. On obtient ainsi

$$\frac{1 - \mu}{(\varepsilon + R)^2} = \frac{1 - \mu}{R^2} (1 + \frac{\varepsilon}{R})^{-2} \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} (1 - \frac{2\varepsilon}{R} + O(\frac{\varepsilon^2}{R^2})) \underset{\varepsilon \rightarrow 0}{=} \frac{1 - \mu}{R^2} -$$

1

et en remplaçant dans l'expression que l'on avait avant on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{R^2} + \frac{2(1 - \mu)\varepsilon}{R^3} + \frac{\mu}{\varepsilon^2} + O(\frac{\varepsilon^2}{R^4}) = 0$$

ce qui se simplifie en

$$\frac{\varepsilon}{R^3} + \frac{(2 - 2\mu)\varepsilon}{R^3} + \frac{\mu}{\varepsilon^2} + O(\frac{\varepsilon^2}{R^4}) = \frac{3 - 2\mu}{R^3} \varepsilon + \frac{\mu}{\varepsilon^2} + O(\frac{\varepsilon^2}{R^4}) = 0$$

et donc à l'ordre 1, cela devient

$$\varepsilon^3 = -\frac{\mu}{3 - 2\mu} R^3 \underset{\mu \rightarrow 0}{\sim} -\frac{\mu}{3} R^3$$

et finalement

$$\varepsilon = -(\frac{\mu}{3})^{\frac{1}{3}} R$$

Ainsi, notre DL en $\frac{\varepsilon}{R}$ est légitime car on observe que $\varepsilon = O_{\mu \rightarrow 0}(\mu^{\frac{1}{3}})$, R étant une constante.

Calcul des points d'équilibre dans le référentiel tournant

Calcul pour L2¹

On a

$$\omega^2 x - \frac{GM_1}{(x + \mu R)^2} - \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x - \frac{GM_1}{(x + \mu R)^2} - \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} - \frac{1 - \mu}{(x + \mu R)^2} - \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \varepsilon + (1 - \mu)R$ on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{(\varepsilon + R)^2} - \frac{\mu}{\varepsilon^2} = 0$$

On peut alors procéder à un développement limité en $\frac{\varepsilon}{R}$. Ce dernier est ici supposé, motivé par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\varepsilon \rightarrow 0$. Le résultat final permettra, ou non, de légitimer notre supposition. On obtient ainsi

$$\frac{1 - \mu}{(\varepsilon + R)^2} = \frac{1 - \mu}{R^2} \left(1 + \frac{\varepsilon}{R}\right)^{-2} = \frac{1 - \mu}{R^2} \left(1 - \frac{2\varepsilon}{R} + O\left(\frac{\varepsilon^2}{R^2}\right)\right) = \frac{1 - \mu}{R^2} -$$

1. On pourra remarquer que les calculs sont identiques à ceux de L1 au signe près du terme en $\frac{\mu}{\varepsilon}$.

et en remplaçant dans l'expression que l'on avait avant on obtient

$$\frac{1 - \mu}{R^2} + \frac{\varepsilon}{R^3} - \frac{1 - \mu}{R^2} + \frac{2(1 - \mu)\varepsilon}{R^3} - \frac{\mu}{\varepsilon^2} + O\left(\frac{\varepsilon^2}{R^4}\right) = 0$$

ce qui se simplifie en

$$\frac{\varepsilon}{R^3} + \frac{(2 - 2\mu)\varepsilon}{R^3} - \frac{\mu}{\varepsilon^2} + O(\varepsilon^2) = \frac{3 - 2\mu}{R^3} \varepsilon - \frac{\mu}{\varepsilon^2} + O(\varepsilon^2) = 0$$

et donc à l'ordre 1, cela devient

$$\varepsilon^3 = \frac{\mu}{3 - 2\mu} R^3 \underset{\mu \rightarrow 0}{\sim} \frac{\mu}{3} R^3$$

et finalement

$$\varepsilon = \left(\frac{\mu}{3}\right)^{\frac{1}{3}} R$$

Ainsi, notre DL en $\frac{\varepsilon}{R}$ est légitime car on observe que $\varepsilon = O_{\mu \rightarrow 0}(\mu^{\frac{1}{3}})$, R étant une constante.

Calcul des points d'équilibre dans le référentiel tournant

Calcul pour L3

On a

$$\omega^2 x + \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en remplaçant ω^2 par son expression on obtient

$$\frac{G(M_1 + M_2)}{R^3} x + \frac{GM_1}{(x + \mu R)^2} + \frac{GM_2}{(x - (1 - \mu)R)^2} = 0$$

et en reconnaissant $\mu = \frac{M_2}{M_1 + M_2}$ on peut simplifier en

$$\frac{x}{R^3} + \frac{1 - \mu}{(x + \mu R)^2} + \frac{\mu}{(x - (1 - \mu)R)^2} = 0$$

et en utilisant le fait que $x = \delta - R$ on obtient

$$\frac{\delta}{R^3} - \frac{1}{R^2} + \frac{1 - \mu}{(\delta - (1 - \mu)R)^2} + \frac{\mu}{(\delta - (2 - \mu)R)^2} = 0$$

On peut alors procéder à deux développements limités en $w := \mu + \frac{\delta}{R}$. Ces derniers sont supposés, motivée par le fait que l'on suppose $\mu \rightarrow 0$ ce qui nous a permis de, qualitativement, supposer $\delta \rightarrow 0$ et donc $w \rightarrow 0$. Le résultat final permettra ici aussi de légitimiser, ou non, notre supposition. On obtient alors

$$\frac{1 - \mu}{(\delta - (1 - \mu)R)^2} = \frac{1 - \mu}{R^2} (1 - w)^{-2} \underset{w \rightarrow 0}{=} \frac{1 - \mu}{R^2} (1 + 2w + o(w)) \underset{w \rightarrow 0}{=} \frac{1}{R^2}$$

5

or on a $(1 - \mu)w = w - \mu w \underset{\mu \rightarrow 0}{=} w + o(w)$ ce qui simplifie le terme précédent en

$$\frac{1 - \mu}{R^2} + \frac{2w}{R^2} + \underset{w \rightarrow 0}{o}(w) = \frac{1 + \mu + \frac{2\delta}{R}}{R^2} + \underset{w \rightarrow 0}{o}(w)$$

On calcule de même

$$\frac{\mu}{(\delta - (2 - \mu)R)^2} = \frac{\mu}{4R^2} (1 - \frac{1}{2}w)^{-2} \underset{w \rightarrow 0}{=} \frac{\mu}{4R^2} (1 + w + o(w)) \underset{w \rightarrow 0}{=} \frac{\mu}{4R^2}$$

On peut remplacer ces deux expressions dans l'expression originale, et obtenir ainsi

$$\frac{\delta}{R^3} - \frac{1}{R^2} + \frac{1 + \mu + 2\delta}{R^2} + \underset{w \rightarrow 0}{o}(w) + \frac{\mu}{4R^2} + \underset{w \rightarrow 0}{o}(w) = \frac{3\delta}{R^3} + \frac{5\mu}{4R^2} + \underset{w \rightarrow 0}{o}(w),$$

ce qui devient à l'ordre 1

$$\frac{\delta}{R^3} = -\frac{5\mu}{12R^2}$$

et donc

$$\delta = -\frac{5\mu}{12}R$$

On peut alors constater que $o(w) = o(\mu + \frac{\delta}{R}) = o(\mu - \frac{5\mu}{12}) = o(\mu)$ ou aussi $o(w) = o(\delta)$. On vérifie ainsi que l'ordre 1 que l'on a écrit est bien celui en δ .

Calcul des points d'équilibre dans le référentiel tournant

Calcul pour L4 et L5

On souhaite d'abord démontrer le passage des relations (??) aux relations (??). On utilise alors $(\frac{1}{u})' = \frac{u'}{u^2}$. On sépare chaque cas suivant si l'on dérive par rapport à x , où y .

Cas de dérivation selon x :

On a dans ce cas $f_1 : (x, y) \mapsto \frac{GmM_1}{r_1} = \frac{GmM_1}{\sqrt{(x+\mu R)^2 + y^2}}$ et

$f_2 : (x, y) \mapsto \frac{GmM_2}{r_2}$, toutes deux des fonctions dérivables selon x sur $\mathbb{R}^*$ comme inverse de fonctions dérivables selon x , à savoir r_1 et r_2 . Ces dernières sont des dérivées partielles selon x

$$\begin{cases} \frac{\partial r_1}{\partial x}(x, y) = \frac{2(x+\mu R)}{2\sqrt{(x+\mu R)^2 + y^2}} = \frac{(x+\mu R)}{r_1(x, y)} \\ \frac{\partial r_2}{\partial x}(x, y) = \frac{(x-(1-\mu)R)}{r_2(x, y)} \end{cases}$$

ce qui nous donne les expressions des dérivées partielles de f_1 et f_2 suivantes (on utilisera la simple notation f' par souci de lisibilité, mais on a bien à faire à des dérivées partielles, le contexte indiquant clairement que c'est selon x ici) :

$$\begin{cases} f'_1(x, y) = \frac{GmM_1 r'_1(x, y)}{r_1^2(x, y)} = \frac{GmM_1}{r_1^2(x, y)} \times \frac{(x+\mu R)}{r_1(x, y)} = \frac{GmM_1(x+\mu R)}{r_1^3(x, y)} \\ f'_2(x, y) = \frac{GmM_2 r'_2(x, y)}{r_2^2(x, y)} = \frac{GmM_2(x-(1-\mu)R)}{r_2^2(x, y)} \end{cases}$$

et assez trivialement, $x \mapsto -\frac{1}{2}m\omega^2(x^2 + y^2)$ admet $x \mapsto -m\omega^2 x$ pour dérivée. On obtient ainsi exactement le terme

pour $\frac{\partial \tilde{U}}{\partial x}$ en sommant chacune des dérivées. Quand au cas selon y , on peut appliquer exactement le même principe. Les calculs seront même plus simple car on voit assez clairement que dans ce cas, $\frac{\partial r_1}{\partial y}(x, y) = \frac{y}{r_1(x, y)}$. Et on obtient bien ainsi les relations (??).

Explicitons le passage de la relation (??) à la relation (??). On commence par imposer la condition d'équilibre, et remplacer par son expression $\omega^2 = \frac{G(M_1 + M_2)}{R^3}$:

$$\begin{cases} 0 = -\frac{Gm(M_1 + M_2)}{R^3}x + \frac{GmM_1(x+\mu R)}{r_1^3} + \frac{GmM_2(x-(1-\mu)R)}{r_2^3} \\ 0 = -\frac{Gm(M_1 + M_2)}{R^3}y + \frac{GmM_1 y}{r_1^3} + \frac{GmM_2 y}{r_2^3} \end{cases}$$

ce qui se simplifie en

$$\begin{cases} \frac{M_1 + M_2}{R^3}x = \frac{M_1(x+\mu R)}{r_1^3} + \frac{M_2(x-(1-\mu)R)}{r_2^3} \\ \frac{M_1 + M_2}{R^3} = \frac{M_1}{r_1^3} + \frac{M_2}{r_2^3} \end{cases}$$

On peut alors diviser par $M_1 + M_2$. On reconnait alors deux termes : $1 - \mu = \frac{M_1}{M_1 + M_2}$ et $\mu = \frac{M_2}{M_1 + M_2}$ ce qui donne alors

$$\begin{cases} \frac{x}{R^3} = \frac{(1-\mu)(x+\mu R)}{r_1^3} + \frac{\mu(x-(1-\mu)R)}{r_2^3} \\ \frac{1}{R^3} = \frac{(1-\mu)}{r_1^3} + \frac{\mu}{r_2^3} \end{cases}$$

et on peut simplifier encore plus la première expression de manière à faire apparaître la deuxième relation, donnant alors

Calcul des points d'équilibre dans le référentiel tournant

$$\frac{x}{R^3} = \left(\frac{1-\mu}{r_1^3} + \frac{\mu}{r_2^3}\right)x + \frac{\mu(1-\mu)R}{r_1^3} - \frac{\mu(1-\mu)R}{r_2^3}$$

ce qui est le résultat voulu.

Explicitons le passage des relations (??) à la relation (??).

$$\frac{x}{R^3} = \frac{x}{R^3} + \mu(1-\mu)\left(\frac{1}{r_1^3} - \frac{1}{r_2^3}\right)R \quad \text{i.e.} \quad \frac{1}{r_1^3} - \frac{1}{r_2^3} = 0$$

ce qui, dans $\mathbb{R}$, nous donne

$$r_1 = r_2$$

et en remplaçant cela dans la deuxième relation de (??) on obtient aussi

$$\frac{1}{R^3} = \frac{1-\mu}{r_1^3} + \frac{\mu}{r_1^3} = \frac{1}{r_1^3}$$

et on a alors dans $\mathbb{R}$ que

$$R = r_1 = r_2$$

d'où la relation (??).

Expression des équations du mouvement

On connaît $\bar{U}$ le potentiel apparent dans R_{NG} . Cela inclue les contributions de F_{ie} , $F_{1/M}$, et $F_{2/M}$. Il manque uniquement la force de Coriolis F_{ic} qui n'est pas conservative donc n'apparaît pas dans le potentiel. Or

$$\vec{F}_{ic} = -2m\vec{\Omega} \times \vec{v} = -2m\Omega\vec{u}_z \times (\dot{x}\vec{u}_x + \dot{y}\vec{u}_y)$$

Alors par principe fondamentale de la dynamique, on déduit que

$$\begin{cases} \ddot{x} - 2\omega\dot{y} = -\frac{1}{m}\bar{U}_x \\ \ddot{y} + 2\omega\dot{x} = -\frac{1}{m}\bar{U}_y \end{cases}$$

qui donne immédiatement les équations du mouvement.

Linéarisation des équations du mouvement

Avec une expansion en série de Taylor à l'ordre 1 en (x_{Li}, y_{Li}) , on obtient

$$\begin{cases} \bar{U}_x(x, y) = \bar{U}_x(x_i, y_i) + \frac{\partial \bar{U}_x}{\partial x}(x_i, y_i) \times (x - x_i) + \frac{\partial \bar{U}_x}{\partial y}(x_i, y_i) \times (y - y_i) \\ \bar{U}_y(x, y) = \bar{U}_y(x_i, y_i) + \frac{\partial \bar{U}_y}{\partial x}(x_i, y_i) \times (x - x_i) + \frac{\partial \bar{U}_y}{\partial y}(x_i, y_i) \times (y - y_i) \end{cases}$$

donc en fixant l'origine spatiale, et du potentiel, en Li , le système linéarisé s'écrit

$$\dot{X} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{v}_x \\ \dot{v}_y \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -\frac{1}{m}\bar{U}_{xx}^i & -\frac{1}{m}\bar{U}_{xy}^i & 0 & 2\omega \\ -\frac{1}{m}\bar{U}_{yx}^i & -\frac{1}{m}\bar{U}_{yy}^i & -2\omega & 0 \end{bmatrix}}_{=:A} \begin{bmatrix} x \\ y \\ v_x \\ v_y \end{bmatrix}$$

Calcul des valeurs et vecteurs propres

Approximation des dérivées secondes des potentiels :

$$\begin{cases} \bar{U}_{xx} = -9m\omega^2 \\ \bar{U}_{yy} = 3m\omega^2 \\ \bar{U}_{xy} = \bar{U}_{yx} = 0 \end{cases}$$

qui nous permet de calculer χ_A le polynôme caractéristique de A . On a

$$\begin{aligned} \chi_A &= \begin{vmatrix} X & 0 & -1 & 0 \\ 0 & X & 0 & -1 \\ -9\omega^2 & 0 & X & -2\omega \\ 0 & 3\omega^2 & 2\omega & X \end{vmatrix} \\ &= X^4 - 2\omega^2 X^2 - 27\omega^2 \\ &= (X - \lambda)(X + \lambda)(X - i\nu)(X + i\nu) \end{aligned}$$

$$\text{où } \begin{cases} \lambda := \omega\sqrt{1+2\sqrt{7}} \in \mathbb{R}_+ \\ \nu := \omega\sqrt{2\sqrt{7}-1} \in \mathbb{R}_+ \end{cases}$$

Equation aux valeurs propres pour déterminer les vecteurs propres :

$$\begin{cases} x_3 = \rho x_1 \\ x_4 = \rho x_2 \\ 9\omega^2 x_1 + 2\omega x_4 = \rho x_3 \\ -3\omega^2 x_2 - 2\omega x_3 = \rho x_4 \end{cases}$$

d'où

$$\begin{cases} (9\omega^2 - \rho^2)x_1 + 2\omega\rho x_2 = 0 \\ (-3\omega^2 - \rho^2)x_2 - 2\omega\rho x_1 = 0 \end{cases}$$

$$\text{donc } P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -\sigma & \sigma & -i\tau & i\tau \\ \lambda & -\lambda & i\nu & -i\nu \\ -\lambda\sigma & -\lambda\sigma & \nu\tau & \nu\tau \end{bmatrix},$$

avec $\sigma = \frac{9\omega^2 - \lambda^2}{2\omega\lambda}$ et $\tau = -\frac{9\omega^2 + \nu^2}{2\omega\nu}$

Orbites périodiques du système linéarisé

Forme des solutions périodiques du système linéarisé :

$$\mathcal{S}_p = \left\{ t \mapsto \begin{bmatrix} R \cos(\nu t - \varphi) \\ \tau R A \sin(\nu t - \varphi) \\ \nu R \sin(\nu t - \varphi) \\ \nu \tau R \cos(\nu t - \varphi) \end{bmatrix}, (R, \varphi) \in \mathbb{R}^2 \right\}$$

et on choisit comme paramètre R l'amplitude de l'orbite selon l'axe Ox . Sans perte de généralité, on peut aussi choisir un déphasage $\varphi = 0$. Cela impose

$$\begin{cases} x(0) = R \\ y(0) = 0 \\ v_x(0) = 0 \\ v_y(0) = \nu \tau R \end{cases}$$

Grâce au module `numpy`, on peut évaluer numériquement A (cf. la partie Python des annexes). On obtient de cette manière que $\nu \tau = 2,37 \cdot 10^{-5}$.

Si Φ et Ψ sont deux matrices fondamentales d'une même équation différentielle. Alors il existe $M \in GL_n(\mathbb{C})$ tel que $\Phi = \Psi M$ et que M soit indépendante de t .

Démonstration : Soit $y_0 \in \mathbb{R}^n$, il existe une unique solution y de l'équation différentielle tel que $y(0) = y_0$. De plus,

$$\forall t \in \mathbb{R}, y(t) = \Phi(t)\Phi(0)^{-1}y_0$$

et de même pour Ψ . Donc

$$\forall t \in \mathbb{R}, \Phi(t)\Phi(0)^{-1}y_0 = \Psi(t)\Psi(0)^{-1}y_0$$

Etant vraie pour tout $y_0 \in \mathbb{R}^n$, on a

$$\forall t \in \mathbb{R}, \Phi(t)\Phi(0)^{-1} = \Psi(t)\Psi(0)^{-1}$$

d'où le résultat en posant $M := \Psi(0)^{-1}\Phi(0)$. M est en effet bien inversible car une matrice fondamentale est inversible pour tout $t \in \mathbb{R}$.

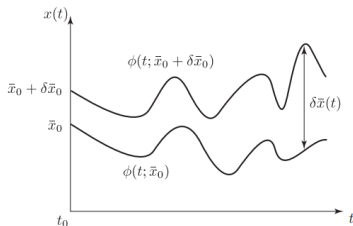
Lien matrice de Monodromie/sensibilité aux C.I.

$$\forall k \in \mathbb{N}^*, \delta x(kT) = M^k \delta x_0$$

Pour $x_0 \in \mathbb{R}^n$ une condition initiale, $t \in \mathbb{R}$ un instant, et $\delta x_0 \in \mathbb{R}^n$ une petite perturbation, on pose

$$\delta x(t) := \eta(t, x_0 + \delta x_0) - \eta(t, x_0)$$

qui représente la variation de la trajectoire en t pour une perturbation δx_0 .



On a que $J\eta(t, x_0) = \Phi(t)$. Et en $t = T$, on obtient bien

$$\delta x(T) = \Phi(T)\delta x_0 = M\delta x_0$$

Théorème de Floquet

Soit $X' = AX$ une équation différentielle où A est à coefficients T -périodiques, et Φ la matrice fondamentale principale du système. Alors il existe $P \in \mathcal{C}_T^1(\mathbb{R}, (M)_n(\mathbb{C}))$ et $B \in (M)_n(\mathbb{C})$ tel que

$$\forall t \in \mathbb{R}, \Phi(t) = P(t)e^{tB}$$

que l'on appelle la *forme normale de Floquet*. En particulier, on appelle $e^{TB} = \Phi(T)$ la *matrice de Monodromie* du système, et *multiplicateurs caractéristiques du système* les valeurs propres de la matrice de Monodromie. Les multiplicateurs caractéristiques sont non nuls et indépendants du choix de Φ et du choix de B .

Avec la forme normale de Floquet, on observe que

$$M^k = (e^{TB})^k = e^{kTB} = P(kT)e^{kTB} = \Phi(kT)$$

Démonstration du théorème de Floquet

Soit $t \in \mathbb{R}$. Par T -périodicité de A , on a

$$\begin{cases} \Phi'(t+T) = A(t+T)\Phi(t+T) = A(t)\Phi(t+T) \\ \det(\Phi(t+T)) = \det(\Phi(t)) \neq 0 \end{cases}$$

i.e. $t \mapsto \Phi(t+T)$ est une matrice fondamentale. Ainsi, il existe $M \in GL_n(\mathbb{C})$, tel que

$$\Phi(t+T) = \Phi(t)M$$

et par surjectivité de l'exponentielle sur $GL_n(\mathbb{C})$, il existe $B \in \mathcal{M}_n(\mathbb{C})$ tel que $M = e^{TB}$.

On pose $\forall t \in \mathbb{R}, P(t) := \Phi(t)e^{-tB}$. Pour tout $t \in \mathbb{R}$,

$$P(t+T) = \Phi(t+T)e^{-(t+T)B} = \Phi(t)Me^{-TB}e^{-tB} = \Phi(t)e^{-tB} = P(t)$$

Fonction de calcul de x_{L1} par dichotomie

```
import src.verlet as vt
import numpy as np

def dichotomy(extrema1, extrema2, precision, system, step=10):
    if extrema1 > extrema2:
        extrema1, extrema2 = extrema2, extrema1
    while abs(extrema2 - extrema1) > precision:
        mid = (extrema1 + extrema2) / 2
        x0, y0, vx0, vy0 = mid, 0, 0, 0
        traj = vt.simulate_trajectory([x0, y0, vx0, vy0], 100, step
                                     , system)
        distance_init = abs(traj[0][0] - extrema1)
        distance_end = abs(traj[-1][0] - extrema1)
        if distance_init < distance_end:
            extrema2 = mid
        else:
            extrema1 = mid
    return (extrema1 + extrema2) / 2
```

Fonctions de calcul des valeurs et vecteurs propres

```
def latexformat_eigenstuff(M):  
    eigM = np.linalg.eig(M)  
    eigenvaluesM = eigM[0]  
    eigenvectorsM = eigM[1]  
  
    latex_matrix_monodromy = pythonMatrix_to_bmatrix(M)  
    latex_eigen_vectors = pythonMatrix_to_bmatrix(eigenvectorsM)  
    print(latex_matrix_monodromy)  
    print(latex_eigen_vectors)  
  
    latex_eigenvalues = "\\{"  
    for i in range(len(eigenvaluesM)):  
        latex_eigenvalues += latex_sci(eigenvaluesM[i])  
        if i != len(eigenvaluesM)-1:  
            latex_eigenvalues += "\\: , "  
    latex_eigenvalues += "\\}"  
    print(latex_eigenvalues)
```

où `pythonMatrix_to_bmatrix` est une fonction simple de formattage.

Calcul de τ et de σ

```
def tau(system, nu):  
    x_L1 = dich.dichotomy(0, system["radius"], precision=1, system=  
        system, step=10)  
    return (-1)*(nu**2 - Ubar_xx_wo_m(x_L1, 0, system))/(2*po.omega(  
        system) * nu)  
  
def sigma(system, lambdavp):  
    x_L1 = dich.dichotomy(0, system["radius"], precision=1, system=  
        system, step=10)  
    return (-1)*(lambdavp**2 - Ubar_xx_wo_m(x_L1, 0, system))/(2*po.  
        omega(system) * lambdavp)
```

Calculs d'informations utiles - Espace de phase *absolue*

```
def phase_space_diag(traj):  
    diag = []  
    for i in range(len(traj)):  
        x = traj[i][0]  
        y = traj[i][1]  
        vx = traj[i][2]  
        vy = traj[i][3]  
        abs_pos = np.sqrt(x**2 + y**2)  
        abs_vel = np.sqrt(vx**2 + vy**2)  
        diag.append([abs_pos, abs_vel])  
        # print(diag[i])  
    return diag
```

Calculs d'informations utiles - Energie

```
def energy(traj,working_system,time_step):  
    t = 0  
    e = []  
    for i in range(len(traj)):  
        x = traj[i][0]  
        y = traj[i][1]  
        vx = traj[i][2]  
        vy = traj[i][3]  
        potential_energy = uo.potential(x,y,working_system)  
        kinetic_energy = 0.5*(vx**2 + vy**2)  
        e.append([t,-2*potential_energy-2*kinetic_energy])  
        t += time_step  
    return e
```

Calculs d'informations utiles - Rayon dans le temps

```
def radius_through_time(traj, time_step):  
    t = 0  
    r = []  
    for i in range(len(traj)):  
        x = traj[i][0]  
        y = traj[i][1]  
        r.append([t, (x**2+y**2)/1e13+4.32e6])  
        t += time_step  
    return r
```


Calculs d'informations utiles - Différence relative 1D

```
def relative_diff_1D(ref, data, steps):  
    relative = []  
    t = 0  
    if (len(ref) != len(data)):  
        print("Error: relative difference not possible due to  
              different array lengths")  
        return [[0,0]]  
    for i in range(len(data)):  
        if (ref[i][1] == 0):  
            print("division by 0")  
            relative.append([ref[i][0],0])  
        else:  
            relative.append([ref[i][0],(data[i][1]-ref[i][1])/ref[i]  
                             ][1]])  
        t += steps  
    return relative
```

Calculs d'informations utiles - Différence relative 2D

```
def relative_diff_2D(ref, data, steps):
    relative = []
    t = 0
    if (len(ref) != len(data)):
        print("Error: relative difference not possible due to
              different array lengths")
        return [[0,0]]
    for i in range(len(data)):
        x = data[i][0]-ref[i][0]
        y = data[i][1]-ref[i][1]
        relative.append([t,np.sqrt(x**2+y**2)/np.sqrt(ref[i][0]**2+
              ref[i][1]**2)])
        t += steps
    return relative
```

Calculs d'informations utiles - Les $\bar{U}_{wz}$

```
def Ubar_xx_wo_m(x,y,system):
    term1 = -po.omega(system)**2
    term2 = G*system["body1"]["mass"]*((r1(x,y,system))**2-3*(x+
        system["barycenter"])**2)/(r1(x,y,system)**5)
    term3 = G*system["body2"]["mass"]*((r2(x,y,system))**2-3*(x-
        system["radius"]+system["barycenter"])**2)/(r2(x,y,system)
        **5)
    return term1 + term2 + term3

def Ubar_xy_wo_m(x,y,system):
    term1 = (system["body1"]["mass"]*(x+system["barycenter"]))*y/(
        r1(x,y,system)**5)
    term2 = (system["body2"]["mass"]*(x-system["radius"]+system["
        barycenter"]))*y/(r2(x,y,system)**5)
    return -3*G*(term1 + term2)

def Ubar_yy_wo_m(x,y,system):
    term1 = -po.omega(system)**2
    term2 = G*system["body1"]["mass"]*((r1(x,y,system))**2-3*y**2)
        /(r1(x,y,system)**5)
    term3 = G*system["body2"]["mass"]*((r2(x,y,system))**2-3*y**2)
        /(r2(x,y,system)**5)
    return term1 + term2 + term3
```

Calculs d'informations utiles - Surfaces équipotentiels

```
def potential(x,y,system):  
    orbital_radius = system["radius"]    # Distance moyenne Terre-  
        Lune en m  
    grav_potential_body1 = -1 * G*body1_mass/(distance(x,y,  
        body1_pos))  
    grav_potential_body2 = -1 * G*body2_mass/(distance(x,y,  
        body2_pos))  
    centrifugal_potential = -0.5 * omega(system)**2*(x**2+y**2)  
    result = grav_potential_body1 + grav_potential_body2 +  
        centrifugal_potential  
    return result  
  
def total_potential(system,surface,step):  
    x = np.linspace(surface[0][0], surface[0][1], step)  
    y = np.linspace(surface[1][0], surface[1][1], step)  
    X,Y = np.meshgrid(x,y)  
    Z = potential(X,Y,system)  
    return (X,Y,Z)
```

Calcul de trajectoires par schéma Runge-Kutta 4 (1/2)

```
def simulate_trajectory(initial_conditions,simulation_time,step,
                        system,inverse_time=False):
    if inverse_time:
        step = -step
        simulation_time = -simulation_time
    dt = step
    t_max = simulation_time
    t = 0
    i = 0
    def eqx(x,y,vy):
        coriolis = 2 * omega * vy
        centrifugal = omega**2 * x
        pull1 = body_pull_x(x,y,body1_pos,body1_mass)
        pull2 = body_pull_x(x,y,body2_pos,body2_mass)
        return pull1 + pull2 + coriolis + centrifugal
    def eqy(x,y,vx):
        coriolis = -2 * omega * vx
        centrifugal = omega**2 * y
        pull1 = body_pull_y(x,y,body1_pos,body1_mass)
        pull2 = body_pull_y(x,y,body2_pos,body2_mass)
        return pull1 + pull2 + coriolis + centrifugal
```

Calcul de trajectoires par schéma Runge-Kutta 4 (2/2)

```
P = [np.array([x0,y0])]
V = [np.array([vx0,vy0])]

def F(P,V):
    return np.array([eqx(P[0],P[1],V[1]), eqy(P[0],P[1],V[0])])

while abs(t) < abs(t_max):
    t += dt
    k1 = F( P[i], V[i])
    k2 = F( P[i] + V[i] * dt/2,
           V[i] + k1 * dt/2)
    k3 = F( P[i] + (V[i]*dt/2) + (k1*(dt/2)**2),
           V[i] + (k2*dt/2))
    k4 = F( P[i] + (dt*V[i]) + (k2*dt**2/2),
           V[i] + k3*dt)
    P.append(P[i] + dt*V[i] + dt**2/6*(k1 + k2 + k3))
    V.append(V[i] + dt/6*(k1 + 2*k2 + 2*k3 + k4))
    i += 1
return P,V
```

Affichage des trajectoires et d'informations utiles

```
def one_traj_relative_origin_display(traj, working_system, time_step
=10, color_palette="plasma"):
    layout = [ // ]
    fig, axes = plt.subplot_mosaic(layout, figsize=(12, 6),
        constrained_layout=True)
    plot_on_ax(axes["traj"], traj, color_palette, isTraj=True)
    plot_on_ax_bodies(axes["traj"], working_system)
    plot_potential(axes["traj"], working_system, pmin=30, levels=50,
        alpha=0.2)
    plot_on_ax(axes["phase"], ps.phase_space_diag(traj),
        color_palette)
    plot_on_ax(axes["energy"], en.energy(traj, working_system,
        time_step), color_palette)
    origin_constant_traj = [traj[0]+[0.1,0,0,0] for i in range(len(
        traj))]
    origin_radius = en.radius_through_time(origin_constant_traj,
        time_step)
    traj_radius = en.radius_through_time(traj, time_step)
    orbit_degenerescence = rd.relative_diff_1D(traj_radius,
        origin_radius, time_step)
    plot_on_ax(axes["relative_radius"], orbit_degenerescence,
        color_palette)
    return orbit_degenerescence
```

Affichage de multiples trajectoires

```
def stacked_trajectories_display(trajs, working_system, time_step=10,
                                color_palette=[]):
    if color_palette == []:
        for i in range(len(trajs)//2):
            color_palette.append("plasma")
        for i in range(len(trajs)//2):
            color_palette.append("viridis")
    layout = [
        ["traj", "phase"]
    ]
    fig, axes = plt.subplot_mosaic(layout, figsize=(12, 6),
                                    constrained_layout=True)
    for i in range(len(trajs)):
        plot_on_ax(axes["traj"], trajs[i], color_palette[i], isTraj=
                    True)
        plot_on_ax(axes["phase"], velocity_traj(trajs[i]),
                    color_palette[i], isTraj=True)
    plot_on_ax_bodies(axes["traj"], working_system)
    plot_potential(axes["traj"], working_system, pmin=30, levels=50,
                    alpha=0.2)
```


Correction itérative d'une orbite du linéarisé (1/4)

```
def linear_orbit_guess_0(amplitude):  
    return np.array([amplitude, 0, 0, -2.37e-5*amplitude])  
  
def find_bracket(system, gamma_0):  
    found = False  
    i = 1  
    while not found:  
        perturbation = np.array([0, 0, 0, 0.1*i*gamma_0[3]])  
        traj_a = rk4.simulate_trajectory(gamma_0+perturbation, 2*T  
                                         /3, 10, system)  
        traj_b = rk4.simulate_trajectory(gamma_0-perturbation, 2*T  
                                         /3, 10, system)  
        x_dot_a = return_state(traj_a)[2]  
        x_dot_b = return_state(traj_b)[2]  
        if x_dot_a * x_dot_b < 0:  
            found = True  
        else:  
            i += 1  
    return [gamma_0[3] - 0.1*i*gamma_0[3], gamma_0[3] + 0.1*i*  
            gamma_0[3]]
```

Correction itérative d'une orbite du linéarisé (2/4)

```
def return_state(traj):
    for i in range(2, len(traj)):
        if traj[i-1][1] < 0 and traj[i][1] > 0:
            alpha = -traj[i-1][1] / (traj[i][1] - traj[i-1][1])
            return traj[i-1] + alpha * (traj[i] - traj[i-1])
    return None

def nearest_in_index(liste, reference, index_considered):
    if len(liste) == 0:
        return -1
    best_i = 0
    best_diff = abs(liste[0][index_considered] - reference)
    for i in range(len(liste)):
        diff = abs(liste[i][index_considered] - reference)
        if diff < best_diff:
            best_i = i
            best_diff = diff
    return best_i
```

Correction itérative d'une orbite du linéarisé (3/4)

```
def dichotomy_vy(amplitude, system, precision):  
    x_L1 = dich.dichotomy(0, system["radius"], precision=1, system=  
        system, step=10)  
    gamma_0 = linear_orbit_guess_0(amplitude) + np.array([x_L1, 0,  
        0, 0])  
    bracket = find_bracket(system, gamma_0)  
    c = (bracket[0]+bracket[1])/2  
    gamma_a = gamma_0.copy()  
    gamma_b = gamma_0.copy()  
    gamma_c = gamma_0.copy()  
    gamma_a[3] = bracket[0]  
    gamma_b[3] = bracket[1]  
    gamma_c[3] = c  
    traj_a = rk4.simulate_trajectory(gamma_a, 2*T/3, 10, system)  
    traj_b = rk4.simulate_trajectory(gamma_b, 2*T/3, 10, system)  
    traj_c = rk4.simulate_trajectory(gamma_c, 2*T/3, 10, system)  
    x_dot_a = return_state(traj_a)[2]  
    x_dot_b = return_state(traj_b)[2]  
    x_dot_c = return_state(traj_c)[2]
```

Correction itérative d'une orbite du linéarisé (4/4)

```
while abs(x_dot_c) > precision:
    if x_dot_c * x_dot_a < 0:
        bracket[1] = c
        x_dot_b = x_dot_c
    else:
        bracket[0] = c
        x_dot_a = x_dot_c

    c = (bracket[0]+bracket[1])/2
    gamma_c[3] = c
    traj_c = rk4.simulate_trajectory(gamma_c, 2*T/3, 10, system
    )
    x_dot_c = return_state(traj_c)[2]
return c
```

Calcul de l'application du linéarisé le long de l'orbite

```
def linearized_system(state, working_system):
    x, y, vx, vy = state
    return np.array([
        [0, 0, 1, 0],
        [0, 0, 0, 1],
        [(-1)*Ubar_xx_wo_m(x, y, working_system), (-1)*Ubar_xy_wo_m(x,
            y, working_system), 0, 2*po.omega(working_system)],
        [(-1)*Ubar_xy_wo_m(x, y, working_system), (-1)*Ubar_yy_wo_m(x,
            y, working_system), -2*po.omega(working_system), 0]
    ])

# Df : renvoie la liste des A_i, A_i etant le linearise en la
# position gamma_i
def Df(orbit, system):
    linearized_map = []
    for i in range(len(orbit)):
        linearized_map.append(linearized_system(orbit[i], system))
    return linearized_map
```

Calcul de la matrice de Monodromie

```
def monodromy_matrix(system, gamma, dt):  
    A = lin.Df(gamma, system)  
    Phi = np.eye(4)  
    for n in range(len(A)-1):  
        A1 = A[n]  
        A2 = A[n]  
        A3 = A[n]  
        A4 = A[n+1]  
        k1 = A1 @ Phi  
        k2 = A2 @ (Phi + dt*k1/2)  
        k3 = A3 @ (Phi + dt*k2/2)  
        k4 = A4 @ (Phi + dt*k3)  
        Phi = Phi + dt*(k1 + 2*k2 + 2*k3 + k4)/6  
    return Phi
```

Intégration d'un sens de la variété instable

```
def unstable_manifold_shadow_1D(starting_state, lambda_i,
    dimensionalization, sim_time, step, system, eps=1e-6):
    ci = starting_state + eps * mult_coef_by_coef(lambda_i,
        dimensionalization)
    return rk4.simulate_trajectory(ci, sim_time, step, system)

def unstable_manifold_sampling(lambda_i, dimensionalization, sim_time
    , step, system, reference_orbit, sampling, eps=1e-6):
    manifold = []
    for i in range(sampling):
        state_i = reference_orbit[i*len(reference_orbit)//sampling]
        print(i*len(reference_orbit)//sampling, state_i)
        manifold.append(unstable_manifold_shadow_1D(state_i,
            lambda_i, dimensionalization, sim_time, step, system, eps))
    return manifold
```

Intégration d'un sens de la variété stable

```
def stable_manifold_shadow_1D(starting_state, lambda_s,
    dimensionalization, sim_time, step, system, eps=1e-6):
    ci = starting_state + eps * mult_coef_by_coef(lambda_s,
        dimensionalization)
    return rk4.simulate_trajectory(ci, sim_time, step, system,
        inverse_time=True)

def stable_manifold_sampling(lambda_s, dimensionalization, sim_time,
    step, system, reference_orbit, sampling, eps=1e-6):
    manifold = []
    for i in range(sampling):
        state_i = reference_orbit[i*len(reference_orbit)//sampling]
        print(i*len(reference_orbit)//sampling, state_i)
        manifold.append(stable_manifold_shadow_1D(state_i, lambda_s,
            dimensionalization, sim_time, step, system, eps))
    return manifold
```